C++ Code

```cpp

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

// Function to display allocation results
void display(int process[], int allocation[], int m,
             int blocks[], int n)
{
    int memory[MAX];
    int allocatedMemory = 0;
    int allocatedCount = 0;
    int totalMemory = 0;
    int freeMemory = 0;

    // Initialize memory blocks
    for (int i = 0; i < n; i++)
    {
        memory[i] = blocks[i];
        totalMemory += blocks[i];
    }

    // Calculate allocated memory
    for (int i = 0; i < m; i++)
    {
        if (allocation[i] != -1)
        {
            memory[allocation[i]] -= process[i];
            allocatedMemory += process[i];
            allocatedCount++;
        }
    }

    // Display process allocation table
    cout << "\n";
    cout << left
         << setw(12) << "Process"
         << setw(15) << "Size"
         << setw(15) << "Block"
         << setw(18) << "Status" << "\n";

    cout << string(60, '-') << "\n";

    for (int i = 0; i < m; i++)
    {
        cout << left
             << setw(12) << ("P" + to_string(i + 1))
```

```
51              << setw(15) << process[i];
52
53          if (allocation[i] != -1)
54          {
55              cout << setw(15) << (allocation[i] + 1)
56                   << setw(18) << "Allocated" << "\n";
57          }
58          else
59          {
60              cout << setw(15) << "-"
61                   << setw(18) << "Not Allocated" << "\n";
62          }
63      }
64
65      // Calculate remaining free memory
66      for (int i = 0; i < n; i++)
67      {
68          freeMemory += memory[i];
69      }
70
71      // Display statistics
72      cout << string(60, '-') << "\n";
73
74      cout << "Successfully allocated processes : "
75           << allocatedCount << "/" << m << "\n";
76
77      cout << "Total memory                     : "
78           << totalMemory << "\n";
79
80      cout << "Allocated memory                 : "
81           << allocatedMemory << "\n";
82
83      cout << "Remaining free memory            : "
84           << freeMemory << "\n";
85
86      cout << fixed << setprecision(2);
87      cout << "Memory utilization               : "
88           << (totalMemory > 0
89               ? (double)allocatedMemory / totalMemory * 100
90               : 0)
91           << "%\n";
92
93      // Display remaining memory in each block
94      cout << "\nRemaining Memory in Each Block\n";
95      cout << string(55, '-') << "\n";
96
97      cout << left
98           << setw(15) << "Block"
99           << setw(20) << "Original Size"
100          << setw(20) << "Free Size" << "\n";
101
102     cout << string(55, '-') << "\n";
103
104     for (int i = 0; i < n; i++)
```

```
    {
        cout << left
             << setw(15) << ("Block " + to_string(i + 1))
             << setw(20) << blocks[i]
             << setw(20) << memory[i] << "\n";
    }

    cout << string(55, '-') << "\n";
}

// FIRST FIT ALGORITHM
void firstFit(int blocks[], int n, int process[], int m)
{
    int allocation[MAX];
    int memory[MAX];

    for (int i = 0; i < n; i++)
        memory[i] = blocks[i];

    for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (memory[j] >= process[i])
            {
                allocation[i] = j;
                memory[j] -= process[i];
                break;
            }
        }
    }

    cout << "\n========== FIRST FIT ==========\n";
    display(process, allocation, m, blocks, n);
}

// BEST FIT ALGORITHM
void bestFit(int blocks[], int n, int process[], int m)
{
    int allocation[MAX];
    int memory[MAX];

    for (int i = 0; i < n; i++)
        memory[i] = blocks[i];

    for (int i = 0; i < m; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++)
    {
        int best = -1;
```

```
159
160         for (int j = 0; j < n; j++)
161         {
162             if (memory[j] >= process[i])
163             {
164                 if (best == -1 || memory[j] < memory[best])
165                     best = j;
166             }
167         }
168
169         if (best != -1)
170         {
171             allocation[i] = best;
172             memory[best] -= process[i];
173         }
174     }
175
176     cout << "\n========== BEST FIT ==========\n";
177     display(process, allocation, m, blocks, n);
178 }
179
180 // WORST FIT ALGORITHM
181 void worstFit(int blocks[], int n, int process[], int m)
182 {
183     int allocation[MAX];
184     int memory[MAX];
185
186     for (int i = 0; i < n; i++)
187         memory[i] = blocks[i];
188
189     for (int i = 0; i < m; i++)
190         allocation[i] = -1;
191
192     for (int i = 0; i < m; i++)
193     {
194         int worst = -1;
195
196         for (int j = 0; j < n; j++)
197         {
198             if (memory[j] >= process[i])
199             {
200                 if (worst == -1 || memory[j] > memory[worst])
201                     worst = j;
202             }
203         }
204
205         if (worst != -1)
206         {
207             allocation[i] = worst;
208             memory[worst] -= process[i];
209         }
210     }
211
212     cout << "\n========== WORST FIT ==========\n";
```

```
    display(process, allocation, m, blocks, n);
}

// MAIN FUNCTION
int main()
{
    int n, m;
    int blocks[MAX], process[MAX];

    cout << "Enter number of memory blocks: ";
    cin >> n;

    if (n <= 0 || n > MAX)
    {
        cout << "Invalid number of memory blocks.\n";
        return 1;
    }

    cout << "Enter memory block sizes:\n";

    for (int i = 0; i < n; i++)
    {
        cin >> blocks[i];

        if (blocks[i] <= 0)
        {
            cout << "Block sizes must be positive.\n";
            return 1;
        }
    }

    cout << "Enter number of processes: ";
    cin >> m;

    if (m <= 0 || m > MAX)
    {
        cout << "Invalid number of processes.\n";
        return 1;
    }

    cout << "Enter process sizes:\n";

    for (int i = 0; i < m; i++)
    {
        cin >> process[i];

        if (process[i] <= 0)
        {
            cout << "Process sizes must be positive.\n";
            return 1;
        }
    }

    // Execute all three algorithms
```

```
    firstFit(blocks, n, process, m);
    bestFit(blocks, n, process, m);
    worstFit(blocks, n, process, m);

    return 0;
}
/*
=========================OUTPUT==============================
Enter number of memory blocks: 5
Enter memory block sizes:
256
300
250
180
160
Enter number of processes: 4
Enter process sizes:
100
356
288
198

========== FIRST FIT ==========

Process     Size           Block          Status
-------------------------------------------------------------
P1          100            1              Allocated
P2          356            -              Not Allocated
P3          288            2              Allocated
P4          198            3              Allocated
-------------------------------------------------------------
Successfully allocated processes : 3/4
Total memory                     : 1146
Allocated memory                 : 586
Remaining free memory            : 560
Memory utilization               : 51.13%

Remaining Memory in Each Block
-----------------------------------------------------
Block          Original Size       Free Size
-----------------------------------------------------
Block 1        256                 156
Block 2        300                 12
Block 3        250                 52
Block 4        180                 180
Block 5        160                 160
-----------------------------------------------------

========== BEST FIT ==========

Process     Size           Block          Status
-------------------------------------------------------------
P1          100            5              Allocated
P2          356            -              Not Allocated
```

```
P3          288            2              Allocated
P4          198            3              Allocated
------------------------------------------------------------
Successfully allocated processes : 3/4
Total memory                     : 1146
Allocated memory                 : 586
Remaining free memory            : 560
Memory utilization               : 51.13%

Remaining Memory in Each Block
-----------------------------------------------------
Block          Original Size       Free Size
-----------------------------------------------------
Block 1        256                 256
Block 2        300                 12
Block 3        250                 52
Block 4        180                 180
Block 5        160                 60
-----------------------------------------------------

========== WORST FIT ==========

Process     Size           Block          Status
------------------------------------------------------------
P1          100            2              Allocated
P2          356            -              Not Allocated
P3          288            -              Not Allocated
P4          198            1              Allocated
------------------------------------------------------------
Successfully allocated processes : 2/4
Total memory                     : 1146
Allocated memory                 : 298
Remaining free memory            : 848
Memory utilization               : 26.00%

Remaining Memory in Each Block
-----------------------------------------------------
Block          Original Size       Free Size
-----------------------------------------------------
Block 1        256                 58
Block 2        300                 200
Block 3        250                 250
Block 4        180                 180
Block 5        160                 160
-----------------------------------------------------
*/
```

C++ Code

```cpp

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

// Function to display frame contents
void displayFrames(int frames[], int n)
{
    for (int i = 0; i < n; i++)
    {
        if (frames[i] == -1)
            cout << setw(8) << "-";
        else
            cout << setw(8) << frames[i];
    }
}

// FIFO Page Replacement
void FIFO(int pages[], int n, int capacity)
{
    int frames[MAX];
    int pointer = 0;
    int faults = 0, hits = 0;

    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    cout << "\n========== FIFO PAGE REPLACEMENT ==========\n";

    cout << left << setw(10) << "Page";
    for (int i = 0; i < capacity; i++)
        cout << setw(8) << ("F" + to_string(i + 1));
    cout << "Status\n";

    cout << string(10 + capacity * 8 + 10, '-') << "\n";

    for (int i = 0; i < n; i++)
    {
        bool found = false;

        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                found = true;
                break;
            }
```

```
        }

        cout << left << setw(10) << pages[i];

        if (found)
        {
            hits++;
            displayFrames(frames, capacity);
            cout << "Hit\n";
        }
        else
        {
            frames[pointer] = pages[i];
            pointer = (pointer + 1) % capacity;
            faults++;

            displayFrames(frames, capacity);
            cout << "Fault\n";
        }
    }

    cout << "Total Page Faults: " << faults << "\n";
    cout << "Total Page Hits  : " << hits << "\n";
}

// LRU Page Replacement
void LRU(int pages[], int n, int capacity)
{
    int frames[MAX];
    int lastUsed[MAX];
    int faults = 0, hits = 0;

    for (int i = 0; i < capacity; i++)
    {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    cout << "\n========== LRU PAGE REPLACEMENT ==========\n";

    cout << left << setw(10) << "Page";
    for (int i = 0; i < capacity; i++)
        cout << setw(8) << ("F" + to_string(i + 1));
    cout << "Status\n";

    cout << string(10 + capacity * 8 + 10, '-') << "\n";

    for (int i = 0; i < n; i++)
    {
        int position = -1;

        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
```

```
            {
                position = j;
                break;
            }
        }

        cout << left << setw(10) << pages[i];

        if (position != -1)
        {
            hits++;
            lastUsed[position] = i;

            displayFrames(frames, capacity);
            cout << "Hit\n";
        }
        else
        {
            int replace = -1;

            for (int j = 0; j < capacity; j++)
            {
                if (frames[j] == -1)
                {
                    replace = j;
                    break;
                }
            }

            if (replace == -1)
            {
                replace = 0;

                for (int j = 1; j < capacity; j++)
                {
                    if (lastUsed[j] < lastUsed[replace])
                        replace = j;
                }
            }

            frames[replace] = pages[i];
            lastUsed[replace] = i;
            faults++;

            displayFrames(frames, capacity);
            cout << "Fault\n";
        }
    }

    cout << "Total Page Faults: " << faults << "\n";
    cout << "Total Page Hits  : " << hits << "\n";
}

// Optimal Page Replacement
```

```cpp
void Optimal(int pages[], int n, int capacity)
{
    int frames[MAX];
    int faults = 0, hits = 0;

    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    cout << "\n========== OPTIMAL PAGE REPLACEMENT ==========\n";

    cout << left << setw(10) << "Page";
    for (int i = 0; i < capacity; i++)
        cout << setw(8) << ("F" + to_string(i + 1));
    cout << "Status\n";

    cout << string(10 + capacity * 8 + 10, '-') << "\n";

    for (int i = 0; i < n; i++)
    {
        bool found = false;

        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                found = true;
                break;
            }
        }

        cout << left << setw(10) << pages[i];

        if (found)
        {
            hits++;
            displayFrames(frames, capacity);
            cout << "Hit\n";
        }
        else
        {
            int replace = -1;

            // Find an empty frame
            for (int j = 0; j < capacity; j++)
            {
                if (frames[j] == -1)
                {
                    replace = j;
                    break;
                }
            }

            // Find the page used farthest in the future
            if (replace == -1)
```

```
            {
                int farthest = -1;

                for (int j = 0; j < capacity; j++)
                {
                    int nextUse = n;

                    for (int k = i + 1; k < n; k++)
                    {
                        if (pages[k] == frames[j])
                        {
                            nextUse = k;
                            break;
                        }
                    }

                    if (nextUse > farthest)
                    {
                        farthest = nextUse;
                        replace = j;
                    }
                }
            }

            frames[replace] = pages[i];
            faults++;

            displayFrames(frames, capacity);
            cout << "Fault\n";
        }
    }

    cout << "Total Page Faults: " << faults << "\n";
    cout << "Total Page Hits  : " << hits << "\n";
}

// Main Function
int main()
{
    int pages[MAX];
    int n, capacity;

    cout << "Enter number of page references: ";
    cin >> n;

    if (n <= 0 || n > MAX)
    {
        cout << "Invalid number of page references.\n";
        return 1;
    }

    cout << "Enter page reference string:\n";
    for (int i = 0; i < n; i++)
        cin >> pages[i];
```

```
267
268     cout << "Enter number of page frames: ";
269     cin >> capacity;
270
271     if (capacity <= 0 || capacity > MAX)
272     {
273         cout << "Invalid number of page frames.\n";
274         return 1;
275     }
276
277     FIFO(pages, n, capacity);
278     LRU(pages, n, capacity);
279     Optimal(pages, n, capacity);
280
281     return 0;
282 }
283 /*
284 ============================OUTPUT============================
285 Enter number of page references: 13
286 Enter page reference string:
287 7 0 1 2 0 3 0 4 2 3 0 3 2
288 Enter number of page frames: 3
289
290 ========== FIFO PAGE REPLACEMENT ==========
291 Page      F1      F2      F3      Status
292 -----------------------------------------
293 7         7       -       -       Fault
294 0         7       0       -       Fault
295 1         7       0       1       Fault
296 2         2       0       1       Fault
297 0         2       0       1       Hit
298 3         2       3       1       Fault
299 0         2       3       0       Fault
300 4         4       3       0       Fault
301 2         4       2       0       Fault
302 3         4       2       3       Fault
303 0         0       2       3       Fault
304 3         0       2       3       Hit
305 2         0       2       3       Hit
306 Total Page Faults: 10
307 Total Page Hits  : 3
308
309 ========== LRU PAGE REPLACEMENT ==========
310 Page      F1      F2      F3      Status
311 -----------------------------------------
312 7         7       -       -       Fault
313 0         7       0       -       Fault
314 1         7       0       1       Fault
315 2         2       0       1       Fault
316 0         2       0       1       Hit
317 3         2       0       3       Fault
318 0         2       0       3       Hit
319 4         4       0       3       Fault
320 2         4       0       2       Fault
```

```
3         4       3       2       Fault
0         0       3       2       Fault
3         0       3       2       Hit
2         0       3       2       Hit
Total Page Faults: 9
Total Page Hits  : 4

========== OPTIMAL PAGE REPLACEMENT ==========
Page      F1      F2      F3      Status
-----------------------------------------
7         7       -       -       Fault
0         7       0       -       Fault
1         7       0       1       Fault
2         2       0       1       Fault
0         2       0       1       Hit
3         2       0       3       Fault
0         2       0       3       Hit
4         2       4       3       Fault
2         2       4       3       Hit
3         2       4       3       Hit
0         2       0       3       Fault
3         2       0       3       Hit
2         2       0       3       Hit
Total Page Faults: 7
Total Page Hits  : 6
*/

```

C++ Code

```cpp

#include <iostream>
#include <iomanip>
#include <algorithm>
#include <string>
using namespace std;

const int MAX = 100;

// Function to calculate total head movement
int calculateMovement(int sequence[], int n, int initialHead)
{
    int movement = 0;
    int current = initialHead;

    for (int i = 0; i < n; i++)
    {
        movement += abs(sequence[i] - current);
        current = sequence[i];
    }

    return movement;
}

// FCFS Algorithm
int FCFS(int requests[], int n, int head)
{
    int sequence[MAX];

    for (int i = 0; i < n; i++)
        sequence[i] = requests[i];

    cout << "\n========== FCFS ==========\n";
    cout << "Head movement sequence: " << head;

    for (int i = 0; i < n; i++)
        cout << " -> " << sequence[i];

    int movement = calculateMovement(sequence, n, head);
    cout << "\nTotal head movement: " << movement << " cylinders\n";

    return movement;
}

// SSTF Algorithm
int SSTF(int requests[], int n, int head)
{
    int sequence[MAX];
    bool visited[MAX] = {false};
    int current = head;
```

```
    int movement = 0;

    cout << "\n========== SSTF ==========\n";
    cout << "Head movement sequence: " << head;

    for (int i = 0; i < n; i++)
    {
        int nearest = -1;
        int minimum = 1000000;

        for (int j = 0; j < n; j++)
        {
            if (!visited[j] && abs(requests[j] - current) < minimum)
            {
                minimum = abs(requests[j] - current);
                nearest = j;
            }
        }

        visited[nearest] = true;
        sequence[i] = requests[nearest];
        movement += abs(sequence[i] - current);
        current = sequence[i];

        cout << " -> " << sequence[i];
    }

    cout << "\nTotal head movement: " << movement << " cylinders\n";

    return movement;
}

// SCAN Algorithm
int SCAN(int requests[], int n, int head, int diskSize, int direction)
{
    int sorted[MAX];

    for (int i = 0; i < n; i++)
        sorted[i] = requests[i];

    sort(sorted, sorted + n);

    int movement = 0;
    int current = head;

    cout << "\n========== SCAN ==========\n";
    cout << "Head movement sequence: " << head;

    if (direction == 1)  // Move towards higher cylinders
    {
        for (int i = 0; i < n; i++)
        {
            if (sorted[i] >= head)
            {
```

```
                movement += abs(sorted[i] - current);
                current = sorted[i];
                cout << " -> " << current;
            }
        }

        if (current != diskSize - 1)
        {
            movement += abs((diskSize - 1) - current);
            current = diskSize - 1;
            cout << " -> " << current;
        }

        for (int i = n - 1; i >= 0; i--)
        {
            if (sorted[i] < head)
            {
                movement += abs(sorted[i] - current);
                current = sorted[i];
                cout << " -> " << current;
            }
        }
    }
    else  // Move towards lower cylinders
    {
        for (int i = n - 1; i >= 0; i--)
        {
            if (sorted[i] <= head)
            {
                movement += abs(sorted[i] - current);
                current = sorted[i];
                cout << " -> " << current;
            }
        }

        if (current != 0)
        {
            movement += current;
            current = 0;
            cout << " -> " << current;
        }

        for (int i = 0; i < n; i++)
        {
            if (sorted[i] > head)
            {
                movement += abs(sorted[i] - current);
                current = sorted[i];
                cout << " -> " << current;
            }
        }
    }

    cout << "\nTotal head movement: " << movement << " cylinders\n";
```

```
159
160     return movement;
161 }
162
163 // C-SCAN Algorithm
164 int CSCAN(int requests[], int n, int head, int diskSize, int direction)
165 {
166     int sorted[MAX];
167
168     for (int i = 0; i < n; i++)
169         sorted[i] = requests[i];
170
171     sort(sorted, sorted + n);
172
173     int movement = 0;
174     int current = head;
175
176     cout << "\n========== C-SCAN ==========\n";
177     cout << "Head movement sequence: " << head;
178
179     if (direction == 1)  // Move towards higher cylinders
180     {
181         for (int i = 0; i < n; i++)
182         {
183             if (sorted[i] >= head)
184             {
185                 movement += abs(sorted[i] - current);
186                 current = sorted[i];
187                 cout << " -> " << current;
188             }
189         }
190
191         if (current != diskSize - 1)
192         {
193             movement += (diskSize - 1) - current;
194             current = diskSize - 1;
195             cout << " -> " << current;
196         }
197
198         // Circular return to cylinder 0
199         if (current != 0)
200         {
201             movement += diskSize - 1;
202             current = 0;
203             cout << " -> " << current;
204         }
205
206         for (int i = 0; i < n; i++)
207         {
208             if (sorted[i] < head)
209             {
210                 movement += abs(sorted[i] - current);
211                 current = sorted[i];
212                 cout << " -> " << current;
```

```cpp
            }
        }
    }
    else  // Move towards lower cylinders
    {
        for (int i = n - 1; i >= 0; i--)
        {
            if (sorted[i] <= head)
            {
                movement += abs(sorted[i] - current);
                current = sorted[i];
                cout << " -> " << current;
            }
        }

        if (current != 0)
        {
            movement += current;
            current = 0;
            cout << " -> " << current;
        }

        // Circular return to the highest cylinder
        if (current != diskSize - 1)
        {
            movement += diskSize - 1;
            current = diskSize - 1;
            cout << " -> " << current;
        }

        for (int i = n - 1; i >= 0; i--)
        {
            if (sorted[i] > head)
            {
                movement += abs(sorted[i] - current);
                current = sorted[i];
                cout << " -> " << current;
            }
        }
    }

    cout << "\nTotal head movement: " << movement << " cylinders\n";

    return movement;
}

// Main Function
int main()
{
    int requests[MAX];
    int n, head, diskSize, direction;

    cout << "Enter number of disk requests: ";
    cin >> n;
```

```cpp

    if (n <= 0 || n > MAX)
    {
        cout << "Invalid number of requests.\n";
        return 1;
    }

    cout << "Enter disk request queue:\n";
    for (int i = 0; i < n; i++)
        cin >> requests[i];

    cout << "Enter initial head position: ";
    cin >> head;

    cout << "Enter total number of cylinders: ";
    cin >> diskSize;

    if (diskSize <= 0 || head < 0 || head >= diskSize)
    {
        cout << "Invalid disk size or head position.\n";
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        if (requests[i] < 0 || requests[i] >= diskSize)
        {
            cout << "Invalid request position.\n";
            return 1;
        }
    }

    cout << "Enter direction (1 = Higher, 0 = Lower): ";
    cin >> direction;

    if (direction != 0 && direction != 1)
    {
        cout << "Invalid direction.\n";
        return 1;
    }

    int fcfs = FCFS(requests, n, head);
    int sstf = SSTF(requests, n, head);
    int scan = SCAN(requests, n, head, diskSize, direction);
    int cscan = CSCAN(requests, n, head, diskSize, direction);

    cout << "\n========== PERFORMANCE COMPARISON ==========\n";
    cout << left << setw(15) << "Algorithm"
         << setw(25) << "Total Head Movement" << "\n";
    cout << string(40, '-') << "\n";

    cout << left << setw(15) << "FCFS"
         << setw(25) << fcfs << "\n";

```

```
    cout << left << setw(15) << "SSTF"
         << setw(25) << sstf << "\n";

    cout << left << setw(15) << "SCAN"
         << setw(25) << scan << "\n";

    cout << left << setw(15) << "C-SCAN"
         << setw(25) << cscan << "\n";

    int minimum = min(min(fcfs, sstf), min(scan, cscan));

    cout << string(40, '-') << "\n";
    cout << "Lowest head movement: " << minimum << " cylinders\n";

    cout << "Most efficient for this input: ";

    if (minimum == fcfs)
        cout << "FCFS";
    else if (minimum == sstf)
        cout << "SSTF";
    else if (minimum == scan)
        cout << "SCAN";
    else
        cout << "C-SCAN";

    cout << "\n";

    return 0;
}
/*
=========================OUTPUT============================
Enter number of disk requests: 8
Enter disk request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter total number of cylinders: 200
Enter direction (1 = Higher, 0 = Lower): 1

========== FCFS ==========
Head movement sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total head movement: 640 cylinders

========== SSTF ==========
Head movement sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movement: 236 cylinders

========== SCAN ==========
Head movement sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total head movement: 331 cylinders

========== C-SCAN ==========
Head movement sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 ->
37
Total head movement: 382 cylinders
```

```
374
375 ========== PERFORMANCE COMPARISON ==========
376 Algorithm      Total Head Movement
377 ---------------------------------------
378 FCFS           640
379 SSTF           236
380 SCAN           331
381 C-SCAN         382
382 ---------------------------------------
383 Lowest head movement: 236 cylinders
384 Most efficient for this input: SSTF
385 */
386
```

C++ Code

```cpp

#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
using namespace std;

// Convert IPv4 address to a 32-bit unsigned integer
bool validateIP(string ip, unsigned int &address)
{
    stringstream ss(ip);
    string part;
    unsigned int value = 0;
    int count = 0;

    while (getline(ss, part, '.'))
    {
        if (part.empty() || part.size() > 3)
            return false;

        for (int i = 0; i < (int)part.size(); i++)
        {
            if (part[i] < '0' || part[i] > '9')
                return false;
        }

        int octet = stoi(part);

        if (octet < 0 || octet > 255)
            return false;

        value = (value << 8) | octet;
        count++;
    }

    if (count != 4)
        return false;

    address = value;
    return true;
}

// Convert integer back to IPv4 format
string intToIP(unsigned int ip)
{
    return to_string((ip >> 24) & 255) + "." +
           to_string((ip >> 16) & 255) + "." +
           to_string((ip >> 8) & 255) + "." +
           to_string(ip & 255);
}
```

```cpp

// Validate subnet mask and check that its bits are contiguous
bool validateMask(string mask, unsigned int &maskValue)
{
    if (!validateIP(mask, maskValue))
        return false;

    bool zeroFound = false;

    for (int i = 31; i >= 0; i--)
    {
        bool bit = (maskValue >> i) & 1U;

        if (!bit)
            zeroFound = true;
        else if (zeroFound)
            return false;
    }

    return true;
}

// Simulate IP address allocation and subnet identification
int main()
{
    string ip1, ip2, subnetMask;
    unsigned int address1, address2, mask;

    cout << "===== IP Address Allocation and Subnet Identification =====\n";

    cout << "\nEnter IP address of Host 1: ";
    cin >> ip1;

    if (!validateIP(ip1, address1))
    {
        cout << "Invalid IPv4 address for Host 1.\n";
        return 1;
    }

    cout << "Enter IP address of Host 2: ";
    cin >> ip2;

    if (!validateIP(ip2, address2))
    {
        cout << "Invalid IPv4 address for Host 2.\n";
        return 1;
    }

    cout << "Enter subnet mask (e.g., 255.255.255.0): ";
    cin >> subnetMask;

    if (!validateMask(subnetMask, mask))
    {
        cout << "Invalid subnet mask.\n";
```

```
        return 1;
    }

    // Calculate network IDs
    unsigned int network1 = address1 & mask;
    unsigned int network2 = address2 & mask;

    // Calculate broadcast address
    unsigned int broadcast1 = network1 | (~mask);
    unsigned int broadcast2 = network2 | (~mask);

    // Display Host 1 information
    cout << "\n========== HOST 1 DETAILS ==========\n";
    cout << "IP Address       : " << ip1 << "\n";
    cout << "Subnet Mask      : " << subnetMask << "\n";
    cout << "Network ID       : " << intToIP(network1) << "\n";
    cout << "Broadcast Address: " << intToIP(broadcast1) << "\n";

    // Display Host 2 information
    cout << "\n========== HOST 2 DETAILS ==========\n";
    cout << "IP Address       : " << ip2 << "\n";
    cout << "Subnet Mask      : " << subnetMask << "\n";
    cout << "Network ID       : " << intToIP(network2) << "\n";
    cout << "Broadcast Address: " << intToIP(broadcast2) << "\n";

    // Compare network IDs
    cout << "\n========== SUBNET IDENTIFICATION ==========\n";

    if (network1 == network2)
    {
        cout << "Both hosts belong to the SAME subnet.\n";
    }
    else
    {
        cout << "Hosts belong to DIFFERENT subnets.\n";
    }

    // Calculate prefix length
    int prefix = 0;

    for (int i = 31; i >= 0; i--)
    {
        if ((mask >> i) & 1U)
            prefix++;
        else
            break;
    }

    cout << "Subnet Prefix: /" << prefix << "\n";

    return 0;
}
/*

```

```
159
160 ================================OUTPUT================================
161 ===== IP Address Allocation and Subnet Identification =====
162
163 Enter IP address of Host 1: 192.168.1.10
164 Enter IP address of Host 2: 192.168.1.20
165 Enter subnet mask (e.g., 255.255.255.0): 255.255.255.0
166
167 ========== HOST 1 DETAILS ==========
168 IP Address       : 192.168.1.10
169 Subnet Mask      : 255.255.255.0
170 Network ID       : 192.168.1.0
171 Broadcast Address: 192.168.1.255
172
173 ========== HOST 2 DETAILS ==========
174 IP Address       : 192.168.1.20
175 Subnet Mask      : 255.255.255.0
176 Network ID       : 192.168.1.0
177 Broadcast Address: 192.168.1.255
178
179 ========== SUBNET IDENTIFICATION ==========
180 Both hosts belong to the SAME subnet.
181 Subnet Prefix: /24
182 */
```